

University of Peradeniya
Faculty of Engineering
Department of Computer Engineering

Design Manual

System Architecture & Engineering Specification

Mountain Climber Health & GPS Tracker
Third Year Design Project (3YP)
Group 23

Team Members

Sahan Jayasundara	E/21/198
Prabash Rathnayaka	E/21/328
Pasan Sandeep	E/21/353

Academic Supervisors

Dr. Isuru Nawinne
Ms. Yashodha Vimukthi

September 2026

Contents

1	Introduction	7
1.1	Project Motivation	7
1.2	Problem Statement	7
1.3	Project Objectives	7
1.4	Project Scope	8
1.4.1	Inclusions	8
1.4.2	Exclusions	8
1.5	Document Structure Overview	9
2	System Architecture	10
2.1	Overall Architecture Overview	10
2.1.1	Layered Architecture	10
2.2	System Block Diagram	10
2.3	Data Flow Architecture	11
2.4	Deployment Strategy in Mountainous Terrain	12
3	Hardware Design	13
3.1	Component Specifications	13
3.2	Climber Main Device	13
3.2.1	Design Rationale	13
3.2.2	Electronic Schematic & PCB Layout	14
3.2.3	Power Supply & PCB Considerations	14
3.3	Wristband Device	14
3.3.1	Design Rationale	14
3.3.2	Electronic Schematic	15
3.4	Repeater Node	15
3.4.1	Design Rationale	15
3.4.2	Electronic Schematic	15
3.5	Base Camp Unit	15
3.5.1	Design Rationale	15
3.5.2	Electronic Schematic	15
4	Communication Protocol Design	17
4.1	LoRa Mesh Protocol	17
4.1.1	LoRa Radio Settings	17
4.1.2	Packet Format	17

4.1.3	Message Types	17
4.1.4	Relay and Routing Logic	18
4.2	BLE Protocol	18
4.3	WiFi and HTTP Protocol	19
4.4	Serial JSON Protocol	19
5	Firmware Design	20
5.1	Operational States	20
5.1.1	Main Climber Device State Machine	20
5.1.2	Wristband State Machine	21
5.2	Key Algorithms	21
5.2.1	Distance Calculation (Haversine Formula)	21
5.2.2	Health Threshold Detection	22
5.2.3	NMEA GPS Sentence Parsing	22
5.2.4	LoRa Packet Assembly and Disassembly	22
5.3	System Configurations	23
5.3.1	Interrupt Handling Strategy	23
5.3.2	Watchdog Timer Configuration	23
5.3.3	Power Management	23
6	Mobile Application Design	24
6.1	Architecture	24
6.1.1	State Management	24
6.2	BLE Scanning and Connection Flow	24
6.2.1	Error Handling and Reconnection Strategy	25
6.3	Key Screens and UI Design Principles	25
6.3.1	Home Screen	25
6.3.2	Vitals Screen	25
6.3.3	Map Screen	25
6.3.4	Messages Screen	27
6.4	Platform-Specific Configurations	27
6.4.1	Android Manifest	27
6.4.2	iOS Info.plist	27
7	Dashboard Design	28
7.1	Application Architecture	28
7.1.1	Threading Model	28
7.2	Real-Time Map Rendering	29
7.2.1	Coordinate Transformation	29
7.2.2	Climber Marker Rendering	29
7.3	SOS Alert Workflow	29
7.4	System Features	29
7.4.1	Two-Way Messaging	29
7.4.2	CSV Export Logic	29
7.4.3	Packaging and Deployment	29

8	Web Portal Design	31
8.1	Architecture	31
8.1.1	Server vs. Client Component Strategy	31
8.2	Supabase Integration	31
8.3	Authentication Flow	32
8.4	Key Portal Features	32
8.4.1	E-Commerce and Product Management	32
8.4.2	Device Registration Workflow	32
8.4.3	Firmware Downloads	32
9	Database Design	34
9.1	Entity-Relationship Model	34
9.2	Table Definitions and SQL Schema	35
9.2.1	SQL Initialization Script	35
9.3	Row-Level Security (RLS) Policies	36
9.3.1	Profiles Policy	36
9.3.2	Orders and Devices Policy	37
9.3.3	Public Access Policies	37
10	Testing & Validation	38
10.1	LoRa Range Testing	38
10.1.1	Test Methodology	38
10.1.2	Results	38
10.2	Sensor Accuracy Validation	38
10.2.1	MAX30102 Health Sensor	40
10.2.2	NEO-6M GPS Module	40
10.3	BLE Connection Reliability	40
10.4	System Integration Testing	41
11	Budget & Bill of Materials	42
11.1	Device Bill of Materials (BOM)	42
11.2	Total Hardware System Cost	43
11.3	Software and Development Costs	43
11.4	Budget Summary	43
12	Limitations & Future Work	45
12.1	Known Limitations	45
12.2	Future Improvements	45
12.2.1	Hardware Enhancements	46
12.2.2	Software & System Enhancements	46
A	Pin Configuration Tables	47
A.1	Climber Main Device (ESP32)	47
A.2	Wristband Device (ESP32-C3 / PICO)	47
A.3	Repeater Node (ESP32)	47
A.4	Base Camp Unit (ESP32)	47

B	LoRa Packet Format Specification	49
B.1	General Packet Structure	49
B.2	Message Types and Payload Definitions	49
B.2.1	0x01: SOS Alert	49
B.2.2	0x02: Routine GPS Update	49
B.2.3	0x03: Health Vitals	50
B.2.4	0x04: Text Message	50
B.3	Example Hex Dumps	50
C	API Reference	51
C.1	Base Camp Dashboard (Flask REST API)	51
C.1.1	GET /	51
C.1.2	GET /get_data	51
C.1.3	POST /send_message	52
C.1.4	GET /export_csv	52
C.2	Summit Gear Web Portal (Next.js Server Actions)	52
C.2.1	registerDevice(serialNum, userId)	52
C.2.2	createOrder(cart: CartItem[], userId: string)	52
C.2.3	getProducts(category?: string)	53
C.2.4	uploadFirmware(file: FormData, version: string)	53

List of Figures

2.1	High-Level System Architecture and Interconnections	11
2.2	Detailed Data Flow Diagram	11
2.3	Deployment diagram illustrating the role of Repeaters in overcoming mountain topology	12
3.1	Climber Main Device Electronic Schematic	14
3.2	Wristband Device Electronic Schematic	15
3.3	Repeater Node Electronic Schematic	15
3.4	Base Camp Unit Electronic Schematic	16
4.1	LoRa Packet Structure	18
5.1	State Machine Diagram for Main Climber Device	20
5.2	State Machine Diagram for Wristband	21
6.1	BLE Connection Sequence Diagram	25
6.2	Mobile App - Home Screen Interface	26
7.1	Dashboard Threading Architecture	28
7.2	SOS Alert Processing Flowchart	30
8.1	Authentication Sequence Diagram	32
8.2	Summit Gear E-Commerce Interface	33
9.1	Entity-Relationship Diagram of the Supabase PostgreSQL Schema.	34
10.1	LoRa Range Testing Setup in Field Conditions	39
B.1	Binary LoRa Packet Structure	49

List of Tables

10.1 LoRa Range Testing Results (Transmitter Power: 20 dBm, BW: 125kHz) .	40
10.2 MAX30102 Accuracy Comparison (Resting State, N=50 samples)	40
11.1 Bill of Materials for a Single Climber Main Device	42
11.2 Hardware Cost for a Standard Expedition Setup	43
11.3 Development and Infrastructure Costs	43
11.4 Grand Total Budget Summary	44

Chapter 1

Introduction

1.1 Project Motivation

Mountaineering and high-altitude hiking are physically demanding activities fraught with environmental hazards. Every year, numerous accidents and fatalities occur due to unpredictable weather, sudden health complications such as Acute Mountain Sickness (AMS), and getting lost in remote terrains. A critical factor exacerbating these situations is the lack of reliable, off-grid communication. Conventional cellular networks are typically unavailable in mountainous regions, meaning that when an emergency strikes, climbers cannot promptly call for help. This results in significantly delayed search-and-rescue (SAR) operations, drastically reducing the chances of survival for the victims. The Mountain Climber Health and GPS Tracker project was conceived to address this critical gap by providing a robust, standalone communication and health monitoring system that operates independently of traditional infrastructure.

1.2 Problem Statement

The core problem addressed by this project is the inability to reliably monitor the vital signs of mountaineers and track their geographical locations in areas devoid of cellular coverage. When a climber faces distress—whether due to physical injury, cardiac issues, or spatial disorientation—there is currently no reliable method for them to automatically alert their base camp or fellow climbers. The lack of an integrated system that combines real-time health telemetry, continuous GPS tracking, and decentralized long-range communication severely hampers the effectiveness and timeliness of emergency response in alpine environments.

1.3 Project Objectives

To address the aforementioned challenges, this project aims to achieve the following specific objectives:

- **Real-Time Health Monitoring:** Continuously measure and record critical vital signs, including heart rate and blood oxygen saturation (SpO_2), using a wearable

device to detect early signs of physiological distress.

- **Continuous GPS Tracking:** Accurately track the climber's geographical coordinates in real-time, allowing base camp operators to monitor their exact location and movement trajectory.
- **Off-Grid LoRa Mesh Communication:** Establish a reliable, low-power, long-range communication network using LoRa technology, configured in a mesh topology to overcome the line-of-sight limitations typical of mountainous terrains.
- **Automated Distress Signaling:** Implement an automated SOS alert mechanism that triggers immediately if a climber's vital signs fall outside safe thresholds or if they manually initiate a panic sequence.
- **Cross-Platform Visualization:** Develop intuitive user interfaces, including a mobile application for climbers and a comprehensive desktop dashboard for base camp operators, to visualize data and facilitate communication.

1.4 Project Scope

1.4.1 Inclusions

The scope of this project encompasses the design, development, and integration of both hardware and software components. This includes:

- The design and fabrication of the Climber Main Device, Wristband, Repeater nodes, and Base Camp Unit using ESP32 microcontrollers.
- Implementation of a custom LoRa communication protocol utilizing SX1278 transceivers at 433 MHz.
- Development of a BLE communication link between the Wristband and the user's smartphone.
- Development of a cross-platform mobile application using Flutter/Dart for local monitoring and messaging.
- Creation of a Python Flask-based local dashboard for the base camp, featuring real-time maps and health data visualization.
- A web portal built with Next.js and Supabase for device registration and e-commerce functionalities related to mountaineering gear.

1.4.2 Exclusions

The following aspects are explicitly excluded from the current scope of the project:

- Integration with global satellite communication networks (e.g., Iridium or Inmarsat).

- Clinical certification of the health monitoring sensors; the devices are intended for informational and preliminary warning purposes, not as certified medical equipment.
- Mass production and commercial packaging of the hardware devices.

1.5 Document Structure Overview

This design manual is organized to provide a comprehensive understanding of the system's architecture and engineering details:

- **Chapter 2: System Architecture** details the overall structural design, data flow, and deployment strategy of the system.
- **Chapter 3: Hardware Design** provides an in-depth look at the electronic components, circuit designs, and power management for each physical device.
- **Chapter 4: Communication Protocols** explains the proprietary data formats and communication standards implemented across LoRa, BLE, WiFi, and Serial interfaces.

Chapter 2

System Architecture

2.1 Overall Architecture Overview

The Mountain Climber Health and GPS Tracker system is built upon a multi-tiered architecture that seamlessly integrates edge computing devices, local communication meshes, and centralized visualization platforms. The architecture is designed to be highly resilient, ensuring continuous operation even in completely off-grid environments. It is broadly categorized into three distinct layers: the Hardware Layer, the Communication Layer, and the Application Layer.

2.1.1 Layered Architecture

- **Hardware Layer:** This is the physical foundation of the system, comprising the edge devices worn by the climbers (Wristband and Main Device), the intermediate infrastructure (Repeaters), and the receiving end (Base Camp Unit). These devices are responsible for data acquisition (GPS, health vitals) and initial data processing.
- **Communication Layer:** Acting as the nervous system, this layer handles all data transmission between the hardware nodes. It utilizes a combination of Bluetooth Low Energy (BLE) for short-range personal area networks, a LoRa mesh network for long-range inter-node communication, and USB Serial for direct data transfer to the dashboard.
- **Application Layer:** This layer consists of the software interfaces used by the end-users. It includes the Flutter-based mobile application used by the climbers and the Python Flask desktop dashboard used by the base camp operators to monitor the expedition.

2.2 System Block Diagram

The following diagram illustrates the high-level interconnection between all system components.

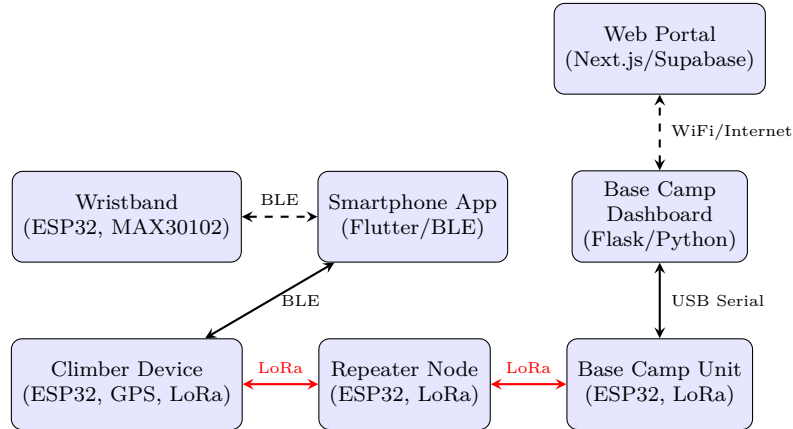


Figure 2.1: High-Level System Architecture and Interconnections

Info

The system operates completely independently of cellular networks during an expedition. Internet connectivity is only required for the Web Portal, which is used pre- and post-expedition for device registration and data sync.

2.3 Data Flow Architecture

Data flows through the system in two primary pipelines: health telemetry and geographical tracking/messaging.

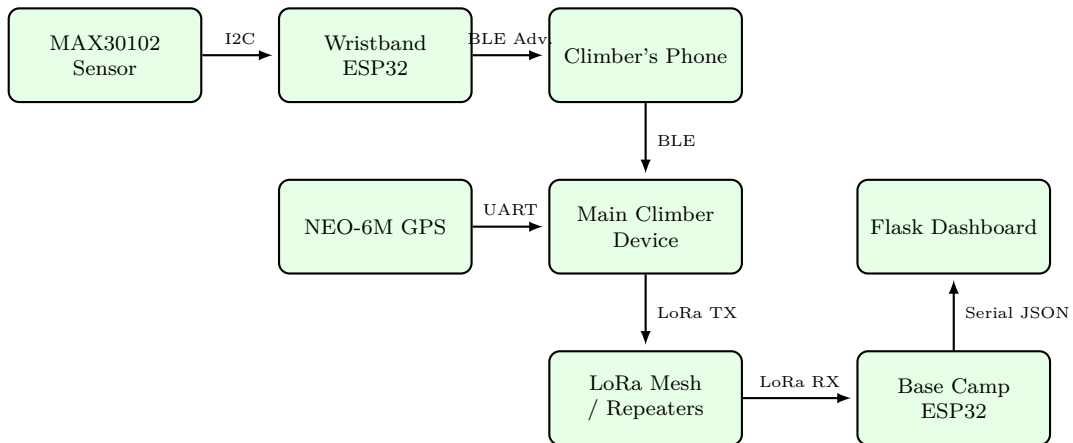


Figure 2.2: Detailed Data Flow Diagram

The data flow begins at the edges:

1. **Health Data:** The MAX30102 sensor on the Wristband reads raw photoplethysmogram (PPG) data, which the ESP32 processes to calculate Heart Rate and SpO₂. This data is advertised via BLE.
2. **Local Aggregation:** The climber's mobile phone app scans for these BLE advertisements, displaying the vitals locally.

3. **Transmission Prep:** The Main Climber Device, which is paired via BLE to the phone and directly reading GPS data via UART, constructs data packets containing the GPS coordinates, health vitals (received via the phone or direct BLE), and system status.
4. **Long-Range Transmission:** These packets are broadcast over the 433 MHz LoRa interface.
5. **Relay:** Repeater nodes pick up the LoRa packets and re-broadcast them, extending the range over rough terrain.
6. **Reception & Display:** The Base Camp Unit receives the LoRa packets, formats them into a serialized JSON structure, and pushes them over a USB serial connection (at 115200 baud) to the laptop running the Python Flask dashboard, which renders the data on an HTML5 canvas map.

2.4 Deployment Strategy in Mountainous Terrain

The physical deployment of the hardware is critical to the system's success. LoRa, operating at 433 MHz, requires line-of-sight (LoS) or near-LoS for maximum range. In a mountain setting, valleys and ridges obstruct radio waves.

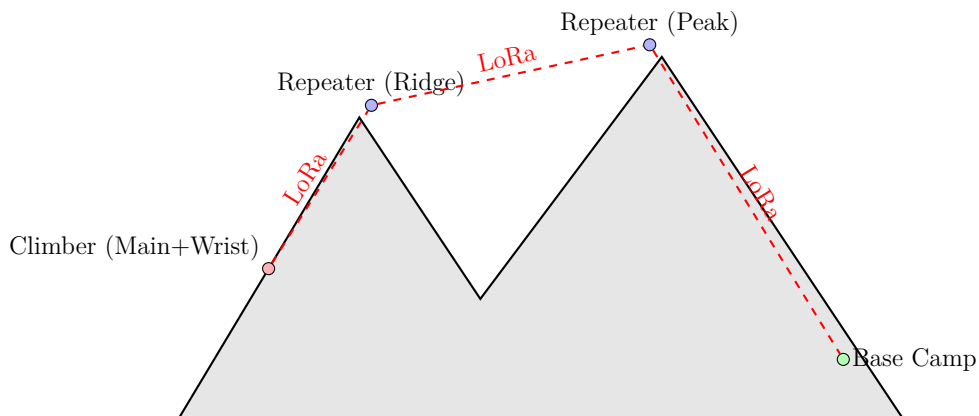


Figure 2.3: Deployment diagram illustrating the role of Repeaters in overcoming mountain topology

To overcome this, **Repeater Nodes** are strategically placed at high elevation points (e.g., ridges or peaks) during the ascent. These nodes act as relay stations. The base camp is typically located in a valley, and the climber may be on the other side of a ridge. The repeater on the ridge ensures continuous communication by acting as a bridge between the climber and the base camp.

Chapter 3

Hardware Design

This chapter details the hardware architecture, component selection, circuit design, and power management strategies for the four distinct physical devices comprising the system.

3.1 Component Specifications

Across the devices, standard modules were chosen for reliability, availability, and community support:

- **ESP32 Microcontroller:** The core processing unit. Features a dual-core Tensilica LX6 microprocessor running at 240 MHz, 520 KB internal SRAM, and integrated Wi-Fi (802.11 b/g/n) and Bluetooth (v4.2 BR/EDR and BLE). Operating voltage is 3.3V.
- **MAX30102 Sensor:** An integrated pulse oximetry and heart-rate monitor module. It communicates via I2C and operates strictly at 3.3V.
- **NEO-6M GPS Module:** A cost-effective, high-performance GPS receiver with a built-in ceramic antenna. It communicates via UART (default 9600 baud rate).
- **SX1278 LoRa Transceiver:** A long-range wireless transceiver operating at the 433 MHz ISM band. It interfaces with the ESP32 via SPI and is crucial for the mesh network.

3.2 Climber Main Device

The Climber Main Device acts as the central hub for the user, aggregating local data and handling long-range communications.

3.2.1 Design Rationale

This device requires significant processing capability to manage BLE connections with the phone, UART communication with the GPS, and SPI communication with the LoRa module simultaneously. The ESP32 is perfectly suited for this due to its dual-core nature and RTOS support.

3.2.2 Electronic Schematic & PCB Layout

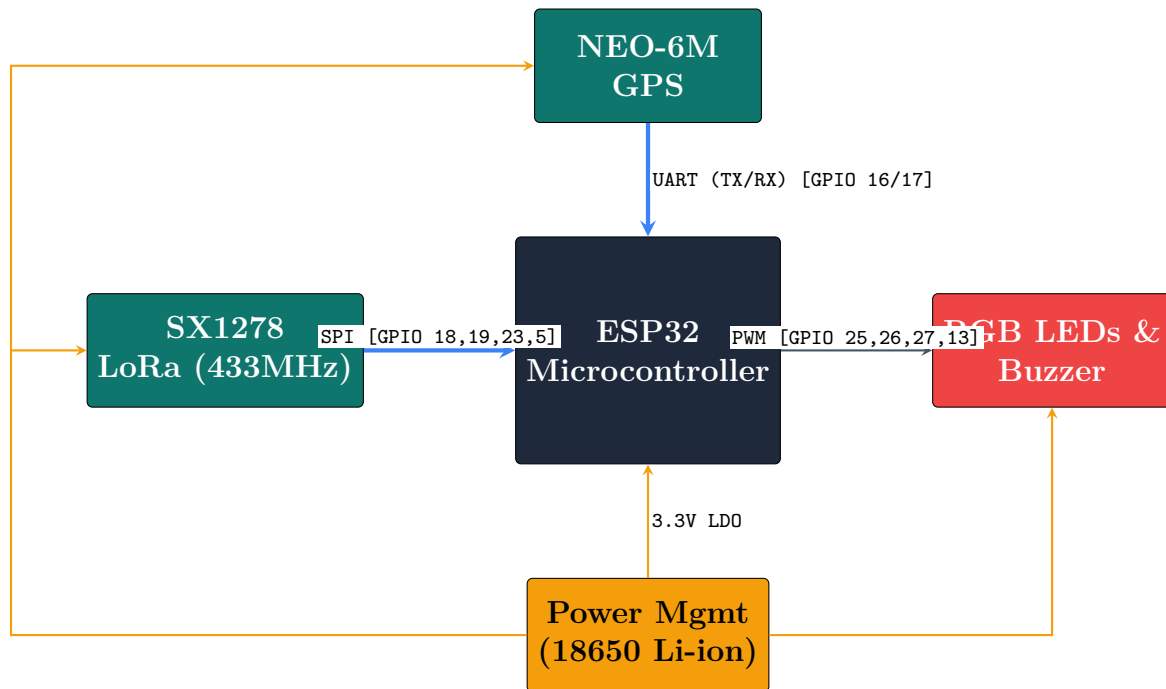


Figure 3.1: Climber Main Device Electronic Schematic

3.2.3 Power Supply & PCB Considerations

Powered by a single 18650 Lithium-ion battery (3.7V nominal, 3400mAh). A low-dropout (LDO) linear regulator (AMS1117-3.3) steps the battery voltage down to the 3.3V required by the ESP32 and peripherals.

PCB Trace Routing:

- **SPI Integrity:** The traces for the SX1278 SPI bus (SCK, MISO, MOSI) are kept under 30mm to prevent signal degradation and crosstalk. Right-angle bends are avoided to maintain impedance continuity.
- **RF Antenna Path:** The trace connecting the SX1278 ANT pin to the SMA connector is designed as a 50Ω coplanar waveguide with ground shielding.
- **Thermal Dissipation:** Large copper pours are utilized around the AMS1117 regulator to act as a heat sink.

3.3 Wristband Device

3.3.1 Design Rationale

The wristband requires a compact form factor. It dedicatedly reads health vitals and advertises them over BLE, allowing it to use a smaller battery footprint.

3.3.2 Electronic Schematic



Figure 3.2: Wristband Device Electronic Schematic

3.4 Repeater Node

3.4.1 Design Rationale

The Repeater node is essentially a stripped-down version of the main device, consisting only of the ESP32, SX1278 LoRa module, and a larger battery capacity (dual 18650s in parallel) since it must remain active for the duration of the expedition on a remote ridge.

3.4.2 Electronic Schematic



Figure 3.3: Repeater Node Electronic Schematic

3.5 Base Camp Unit

3.5.1 Design Rationale

The Base Camp Unit mirrors the Repeater hardware but operates in a different software mode. It requires a stable serial data connection to the laptop running the Dashboard.

3.5.2 Electronic Schematic

Caution

Ensure that the Base Camp Unit's LoRa antenna is properly connected before powering on via USB. Transmitting LoRa signals without an antenna connected can irreversibly damage the SX1278 transceiver amplifier due to impedance mismatch.

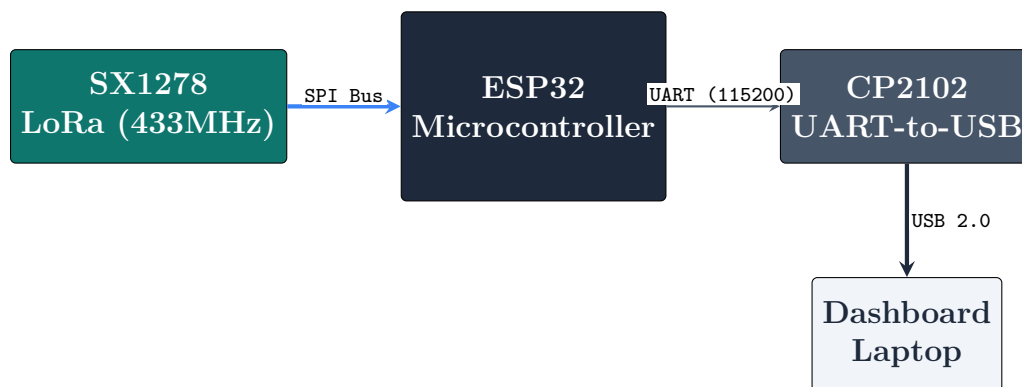


Figure 3.4: Base Camp Unit Electronic Schematic

Chapter 4

Communication Protocol Design

This chapter outlines the proprietary data formats, transmission settings, and protocols developed to ensure reliable data flow across the heterogeneous network interfaces used in the system.

4.1 LoRa Mesh Protocol

The backbone of the system's off-grid communication is the 433 MHz LoRa network. Rather than a simple point-to-point link, the system implements a rudimentary mesh/re-relay logic to overcome the topological challenges of mountainous environments.

4.1.1 LoRa Radio Settings

- **Frequency:** 433 MHz (ISM band suitable for high obstacle penetration).
- **Spreading Factor (SF):** SF7. This was chosen as a compromise. While SF12 offers the maximum range, it drastically increases the Time on Air (ToA), leading to packet collisions in a mesh setup. SF7 provides sufficient range when combined with repeaters while keeping transmission times short.
- **Bandwidth (BW):** 125 kHz. Standard bandwidth balancing range and data rate.
- **Coding Rate (CR):** 4/5. Provides minimal forward error correction, reducing overhead since the mesh logic implements its own packet acknowledgment and retry mechanisms.

4.1.2 Packet Format

All LoRa transmissions adhere to a custom binary packet structure to minimize payload size.

4.1.3 Message Types

The **Type** byte dictates how the payload is parsed:

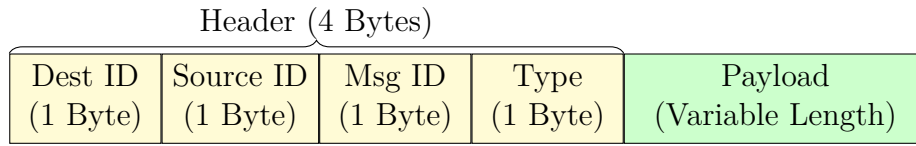


Figure 4.1: LoRa Packet Structure

- **0x01 (SOS)**: Highest priority. Payload contains critical GPS and vital data. Nodes relay this immediately.
- **0x02 (GPS)**: Standard location update.
- **0x03 (HEARTBEAT)**: Sent periodically to update node routing tables and signify device online status.
- **0x04 (TEXT)**: Custom string messages sent from the mobile app to the dashboard, or vice-versa.

4.1.4 Relay and Routing Logic

When a node receives a packet:

1. It checks if the **Dest ID** matches its own ID. If yes, it processes the packet.
2. If not, it checks a local buffer to see if it has recently forwarded this **Msg ID** from this **Source ID**.
3. If not forwarded recently, it rebroadcasts the packet, decrementing a Time-To-Live (TTL) counter (embedded in the payload for standard messages) to prevent infinite loops.

4.2 BLE Protocol

The Wristband communicates with the Climber's phone via Bluetooth Low Energy (BLE). The ESP32 on the wristband acts as a BLE Peripheral, hosting a custom GATT (Generic Attribute Profile) Server.

- **Service UUID:** 4fafc201-1fb5-459e-8fcc-c5c9c331914b
- **Heart Rate Characteristic:** beb5483e-36e1-4688-b7f5-ea07361b26a8 (Notify, Read)
- **SpO2 Characteristic:** 8a7c2936-e8cc-4d37-84be-5bdba349e59d (Notify, Read)

The sensor data is packed into byte arrays and pushed to the phone via GATT notifications at a frequency of 1 Hz.

4.3 WiFi and HTTP Protocol

The Base Camp Unit operates in WiFi SoftAP (Software Access Point) mode if a wireless connection to the dashboard is preferred over USB serial. It hosts a lightweight asynchronous web server.

Listing 4.1: ESP32 SoftAP Configuration Snippet

```
1 WiFi.softAP("BaseCamp_Network", "mountain123");
2 IPAddress IP = WiFi.softAPIP();
```

The HTTP server exposes RESTful endpoints:

- GET /data - Returns the latest JSON array of received LoRa packets.
- POST /send - Accepts JSON payloads from the dashboard to be broadcasted via LoRa to climbers.

4.4 Serial JSON Protocol

The primary and most robust link between the Base Camp ESP32 and the Python Flask Dashboard is a direct USB Serial connection operating at 115200 baud. Data received from the LoRa mesh is encapsulated into JSON objects and printed to the serial port.

Listing 4.2: Example Serial JSON Packet

```
1 {
2   "timestamp": 1694770000,
3   "source_id": "CLIMBER_01",
4   "type": "GPS_VITALS",
5   "data": {
6     "lat": 7.2906,
7     "lng": 80.6337,
8     "hr": 85,
9     "spo2": 96
10  },
11  "rssi": -92
12 }
```

The Flask application runs a background thread that continuously reads the serial port, parses these JSON packets, and emits them to the web frontend using WebSockets.

Chapter 5

Firmware Design

This chapter details the firmware design for the microcontrollers operating within the Mountain Climber Health & GPS Tracker system. The firmware handles critical tasks including sensor data acquisition, wireless communication (BLE and LoRa), power management, and real-time processing.

5.1 Operational States

The firmware on the Main Climber Device and the Wristband operates using finite state machines to ensure deterministic behavior and efficient power utilization.

5.1.1 Main Climber Device State Machine

The Main Climber Device acts as the central hub for a climber, coordinating between the wristband (via BLE) and the mesh network (via LoRa).

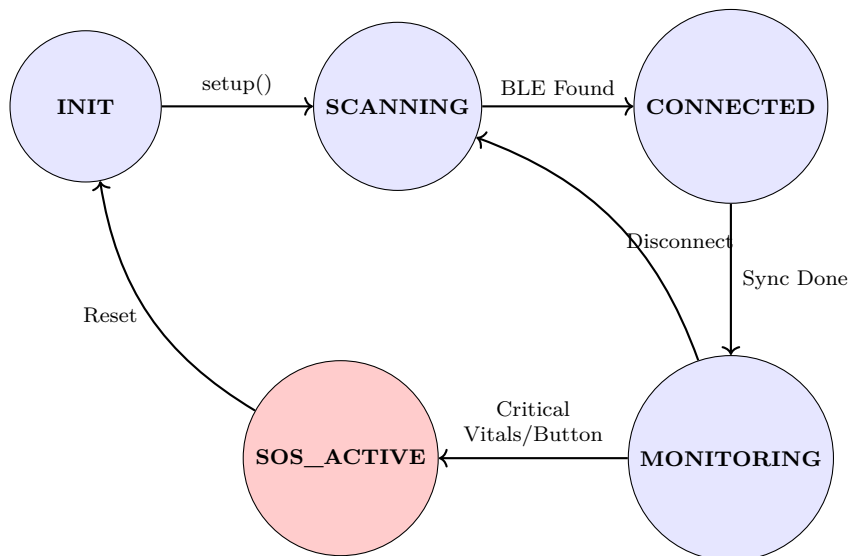


Figure 5.1: State Machine Diagram for Main Climber Device

5.1.2 Wristband State Machine

The wristband's primary role is to acquire health data and transmit it via BLE.

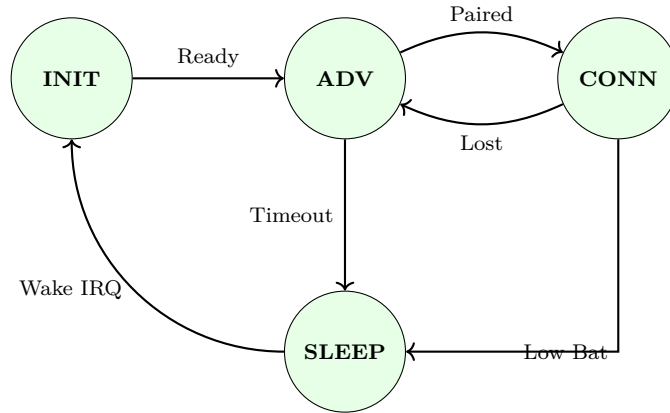


Figure 5.2: State Machine Diagram for Wristband

5.2 Key Algorithms

5.2.1 Distance Calculation (Haversine Formula)

To calculate the distance between climbers or between a climber and the base camp, the firmware utilizes the Haversine formula. This computes the great-circle distance between two points on a sphere given their longitudes and latitudes.

The formula is defined as:

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (5.1)$$

$$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right) \quad (5.2)$$

$$d = R \cdot c \quad (5.3)$$

Where:

- φ_1, φ_2 are the latitudes of point 1 and point 2 (in radians).
- $\Delta\varphi = \varphi_2 - \varphi_1$
- $\Delta\lambda = \lambda_2 - \lambda_1$ (difference in longitude, in radians).
- R is the Earth's radius (mean radius = 6,371 km).

5.2.2 Health Threshold Detection

The firmware continuously evaluates the readings from the MAX30102 sensor. Normal heart rates typically fall between 60 to 100 bpm, while SpO2 levels should ideally remain above 95%. In high-altitude environments, SpO2 can naturally drop, but dangerous hypoxia must be identified.

The threshold algorithm triggers an automatic SOS alert if:

- Heart Rate (HR) < 40 bpm OR HR > 120 bpm
- SpO2 < 90%

Listing 5.1: Health Threshold Checking Algorithm

```
1 void checkHealthThresholds(int hr, int spo2) {
2     bool isEmergency = false;
3
4     if (hr < 40 || hr > 120) {
5         isEmergency = true;
6         logWarning("Abnormal Heart Rate detected!");
7     }
8
9     if (spo2 < 90 && spo2 > 0) { // Ignore 0 which means no
10         // finger attached
11         isEmergency = true;
12         logWarning("Low SpO2 detected!");
13     }
14
15     if (isEmergency) {
16         triggerAutomaticSOS();
17     }
18 }
```

5.2.3 NMEA GPS Sentence Parsing

The NEO-6M GPS module outputs data in NMEA 0183 format. The firmware primarily parses the \$GPRMC (Recommended Minimum Specific GPS/TRANSIT Data) and \$GPGLL (Global Positioning System Fix Data) sentences to extract latitude, longitude, and altitude.

5.2.4 LoRa Packet Assembly and Disassembly

Data transmitted over the 433MHz LoRa network is structured to minimize payload size while ensuring reliable delivery.

Info

The LoRa settings used are: Spreading Factor (SF) 7, Bandwidth (BW) 125kHz, and Coding Rate (CR) 4/5. These settings provide a balanced trade-off between range and data rate.

A typical packet structure includes:

- **Header:** Node ID, Packet Type (SOS, GPS, VITALS, TEXT).
- **Payload:** Formatted data bytes.
- **Checksum:** Simple CRC for integrity verification.

5.3 System Configurations

5.3.1 Interrupt Handling Strategy

Hardware interrupts are utilized for time-critical events to prevent polling overhead.

- **MAX30102 Data Ready:** Triggers an interrupt when a new sample is available, waking the I2C read task.
- **LoRa DIO0:** Triggers on TxDone or RxDone, signaling the communication task.
- **SOS Button:** Configured as a high-priority falling-edge interrupt with hardware debouncing.

5.3.2 Watchdog Timer Configuration

To ensure system reliability, the ESP32 Hardware Watchdog Timer (WDT) is configured with a 5-second timeout. Core tasks (sensor reading, communications) must periodically reset the WDT. If a task hangs due to an I2C lockup or an infinite loop, the WDT resets the microcontroller, ensuring the device recovers in remote environments.

5.3.3 Power Management

Power management is critical for the 18650 Li-ion battery. The firmware employs ESP32's sleep modes:

- **Light Sleep:** Used during idle periods where memory and state are preserved, but CPU clock is gated.
- **Deep Sleep:** Used in the wristband when detached. The device wakes up only via an external interrupt (wake-on-interrupt) triggered by the onboard accelerometer or a physical button press.

Chapter 6

Mobile Application Design

This chapter details the design of the mobile application developed for the Mountain Climber Health & GPS Tracker system. Built using Flutter and Dart, the application provides climbers with real-time access to their health metrics, location data, and communication tools.

6.1 Architecture

The application follows a reactive architecture utilizing the standard Flutter widget tree paradigm combined with a robust state management solution.

6.1.1 State Management

The app employs the *Provider* pattern for state management. This approach separates business logic from UI components, allowing different parts of the application to react to changes in BLE connection status, incoming sensor data, or new messages without requiring widespread widget rebuilds.

6.2 BLE Scanning and Connection Flow

The mobile application acts as a BLE Central device, scanning for and connecting to the Climber Main Device (acting as a BLE Peripheral).

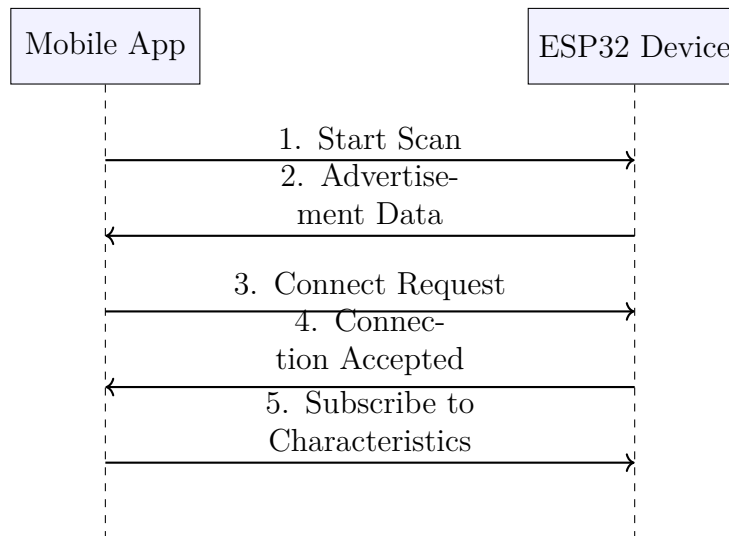


Figure 6.1: BLE Connection Sequence Diagram

6.2.1 Error Handling and Reconnection Strategy

Mountainous environments are prone to interference that can drop BLE connections. The application implements an exponential backoff strategy for reconnection. If the connection drops, the app attempts to reconnect immediately. If that fails, it waits 2 seconds, then 4 seconds, up to a maximum interval, ensuring that battery life is not drained by continuous aggressive polling.

6.3 Key Screens and UI Design Principles

The UI is designed with high contrast and large touch targets, considering that climbers may be operating the app under harsh lighting conditions or while wearing gloves.

6.3.1 Home Screen

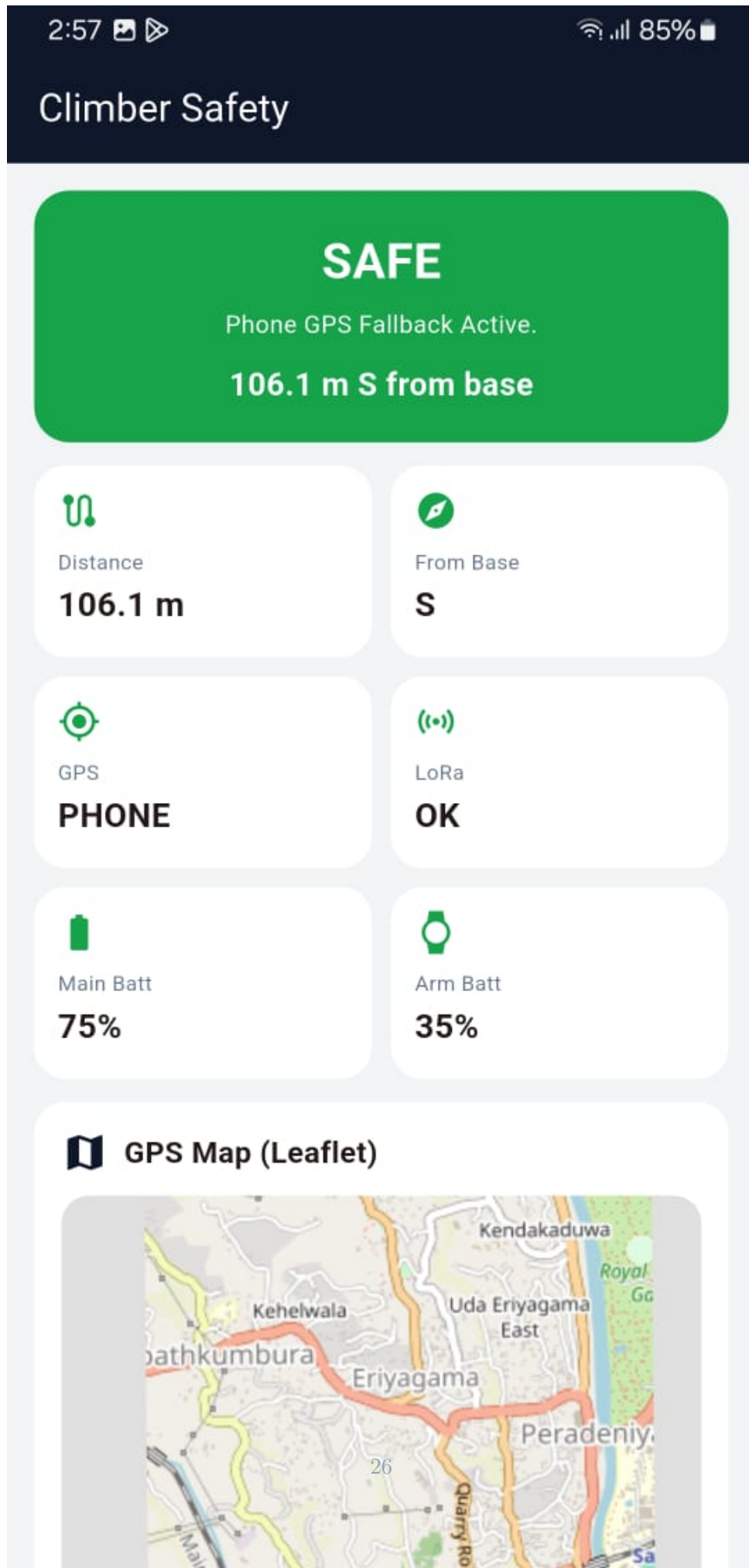
The Home screen serves as a dashboard summarizing the climber's status. It indicates the current BLE connection state and provides quick access to vital metrics and active alerts.

6.3.2 Vitals Screen

Displays real-time readings of Heart Rate, SpO2, and ambient temperature. Gauges and charts are used to show trends over time. Thresholds are color-coded (e.g., green for normal, red for critical).

6.3.3 Map Screen

Utilizes a custom painter in Flutter to render a specialized distance map. This map does not rely on internet map tiles (which are unavailable offline) but plots the relative positions



of the user, the base camp, and other climbers based on received GPS coordinates and Haversine distance calculations.

6.3.4 Messages Screen

Provides a chat-like interface for two-way text messaging. Messages typed here are sent via BLE to the Climber Main Device, which relays them over the LoRa mesh network to the base camp or other climbers.

6.4 Platform-Specific Configurations

To utilize Bluetooth hardware, platform-specific permissions must be explicitly requested.

6.4.1 Android Manifest

The `AndroidManifest.xml` requires permissions for scanning, connecting, and accessing location (historically required for BLE scans on Android).

Listing 6.1: Android BLE Permissions

```

1 <uses-permission android:name="android.permission.BLUETOOTH_SCAN"
  android:usesPermissionFlags="neverForLocation" />
2 <uses-permission android:name="android.permission.
  BLUETOOTH_CONNECT" />
3 <uses-permission android:name="android.permission.
  ACCESS_FINE_LOCATION" />

```

6.4.2 iOS Info.plist

The `iOS Info.plist` must include usage descriptions explaining to the user why Bluetooth is required.

Listing 6.2: iOS Info.plist BLE Usage Description

```

1 <key>NSBluetoothAlwaysUsageDescription</key>
2 <string>This app requires Bluetooth to connect to the climber
  health tracker.</string>
3 <key>NSBluetoothPeripheralUsageDescription</key>
4 <string>This app requires Bluetooth to communicate with the
  tracker.</string>

```

Chapter 7

Dashboard Design

The Base Camp Dashboard is a critical component operated by the expedition monitoring team. It provides a comprehensive overview of all climbers in the network, visualizing their health data, GPS locations, and managing communications.

7.1 Application Architecture

The dashboard is built using Python with the Flask web framework. It operates as a local web server, rendering an interactive frontend while managing serial communications with the Base Camp LoRa Receiver in the backend.

7.1.1 Threading Model

To prevent the web server from blocking while waiting for serial data, the application employs a multi-threaded architecture.

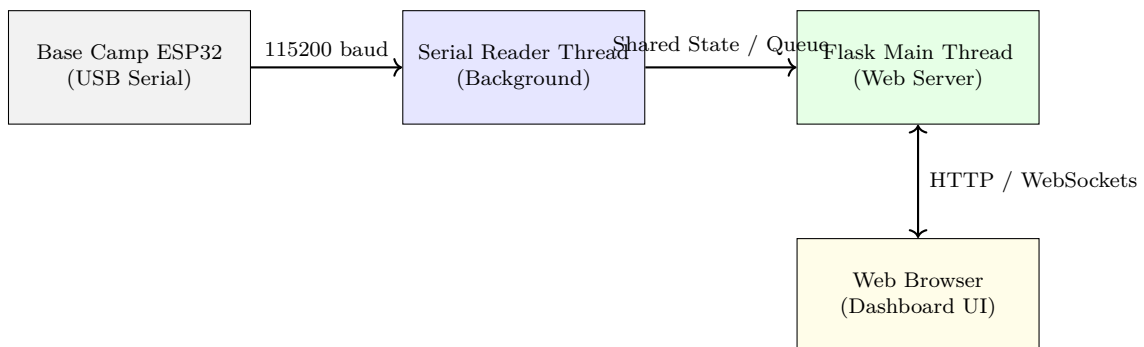


Figure 7.1: Dashboard Threading Architecture

The **Serial Reader Thread** continuously reads JSON payloads from the USB serial port connected to the ESP32. It parses this data and updates a thread-safe shared state object. The **Flask Main Thread** handles incoming HTTP requests from the browser and serves the latest data from the shared state.

7.2 Real-Time Map Rendering

The dashboard features a live tracking map implemented using HTML5 Canvas.

7.2.1 Coordinate Transformation

Since the dashboard must operate completely offline without access to tile servers (like Google Maps or Mapbox), it plots climbers relative to the base camp. The Haversine formula is used to calculate the distance and bearing of each climber relative to the base camp's GPS origin. These polar coordinates are then converted to Cartesian coordinates (x, y) and scaled to fit the HTML5 Canvas dimensions.

7.2.2 Climber Marker Rendering

The canvas dynamically renders markers for each climber.

Tip

Markers are color-coded based on the climber's status: Green for normal, Orange for warning (e.g., missed heartbeat), and flashing Red for an active SOS.

7.3 SOS Alert Workflow

Handling emergencies swiftly is the primary objective of the system.

7.4 System Features

7.4.1 Two-Way Messaging

The dashboard includes a chat interface. Messages typed in the web UI are sent via an API endpoint to the Flask backend, which formats them into a JSON payload and transmits them over the serial port to the Base Camp ESP32. The ESP32 then broadcasts the message over the LoRa network.

7.4.2 CSV Export Logic

For post-expedition analysis, all received data is logged. The backend utilizes Python's built-in `csv` module to append incoming sensor data, GPS coordinates, and timestamps to a daily log file, ensuring no data is lost during the mission.

7.4.3 Packaging and Deployment

The decision was made to embed the HTML, CSS, and JS directly within Python strings (or simple templates) rather than managing a separate frontend build process. This simplifies deployment. The entire Flask application, including the embedded web assets,

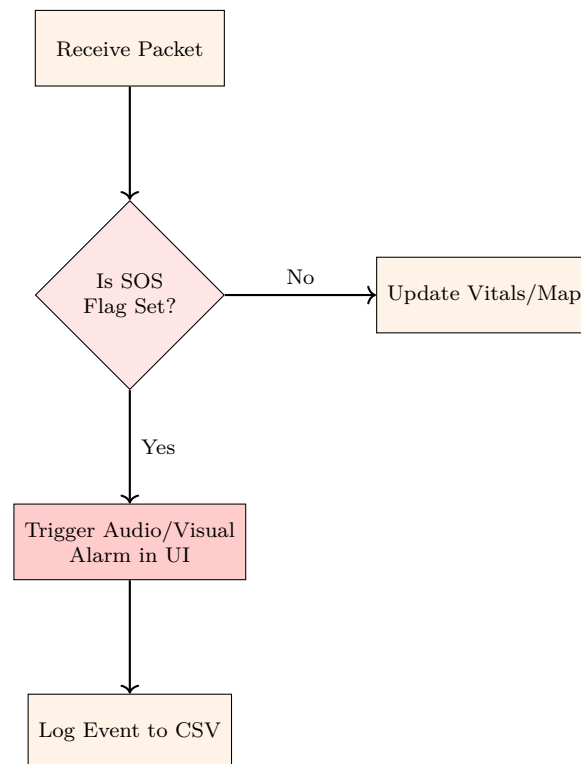


Figure 7.2: SOS Alert Processing Flowchart

is compiled into a single executable binary using **PyInstaller**. This allows expedition staff to run the dashboard on Windows or macOS laptops simply by double-clicking an executable, without needing to install Python or configure environments.

Chapter 8

Web Portal Design

The web portal serves as the commercial and administrative front-end for the Mountain Climber Health & GPS Tracker project, operating under the "Summit Gear" brand. It handles e-commerce for climbing equipment, device registration, and firmware distribution.

8.1 Architecture

The portal is built using **Next.js** utilizing the App Router architecture, backed by **Supabase** for database and authentication services.

8.1.1 Server vs. Client Component Strategy

Next.js 13+ introduced React Server Components. The portal leverages this by:

- **Server Components (Default):** Used for fetching product catalogs, rendering SEO-heavy pages, and handling database queries securely on the server. This reduces the JavaScript bundle size shipped to the client.
- **Client Components:** Explicitly marked with `"use client"`, these are used only where interactivity is required, such as the shopping cart, checkout forms, and real-time dashboard filtering.

8.2 Supabase Integration

Supabase provides a PostgreSQL database and an authentication layer.

- **Server-Side Client:** Used in Next.js API routes and Server Components for secure data fetching (e.g., retrieving user orders) utilizing Server-Side Rendering (SSR).
- **Browser Client:** Used in Client Components for real-time updates (e.g., listening to stock changes) and handling client-side authentication states.

8.3 Authentication Flow

The portal utilizes Supabase Auth for managing users securely.

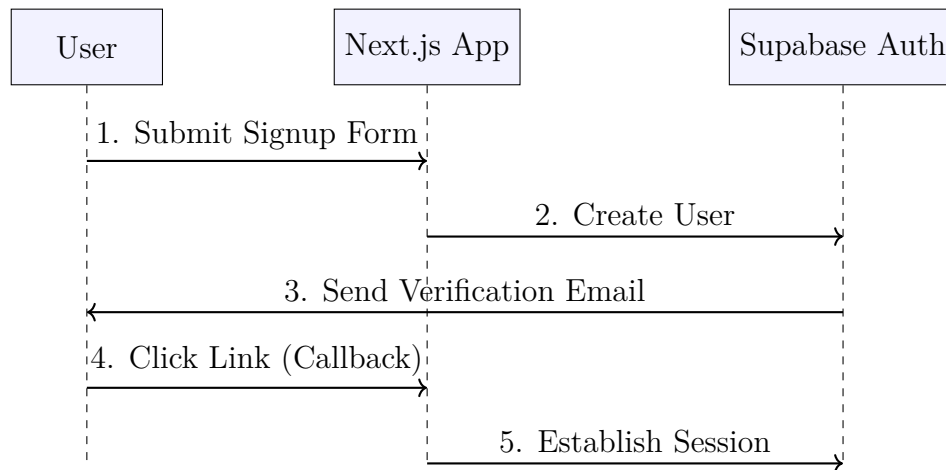


Figure 8.1: Authentication Sequence Diagram

8.4 Key Portal Features

8.4.1 E-Commerce and Product Management

The "Summit Gear" store allows users to browse climbing equipment and the tracker devices. The system implements a standard e-commerce flow: Product Browsing → Cart → Checkout. Administrators can manage product listings, update stock levels, and process incoming orders via a protected admin dashboard.

8.4.2 Device Registration Workflow

When a user purchases a Climber Health Tracker, they must register the device on the portal using a unique serial number. This workflow links the physical device's MAC address and LoRa Node ID to the user's account in the PostgreSQL database. Row-Level Security (RLS) in Supabase ensures that users can only view and manage devices registered to their own account.

8.4.3 Firmware Downloads

The portal provides a secure section for registered users to download the latest OTA (Over-The-Air) firmware binaries. The Next.js backend verifies the user's session and device ownership before granting access to the firmware files stored in Supabase Storage.

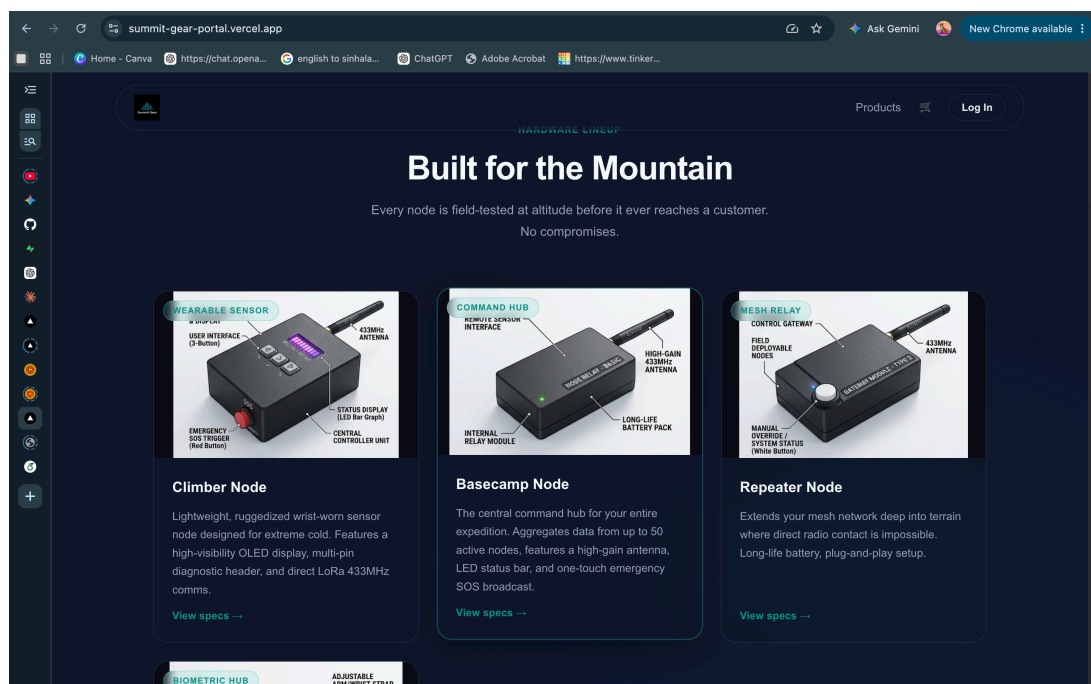


Figure 8.2: Summit Gear E-Commerce Interface

Chapter 9

Database Design

The Summit Gear web portal and device management system rely on a robust, relational database architecture hosted on Supabase, a managed PostgreSQL platform. This chapter details the database schema, entity relationships, security policies, and the SQL statements used for initialization.

9.1 Entity-Relationship Model

The database schema is designed to handle e-commerce operations, user management, and IoT device registration. The core entities include users, products, orders, order items, devices, and firmware releases. Figure 9.1 illustrates the relationships between these entities.

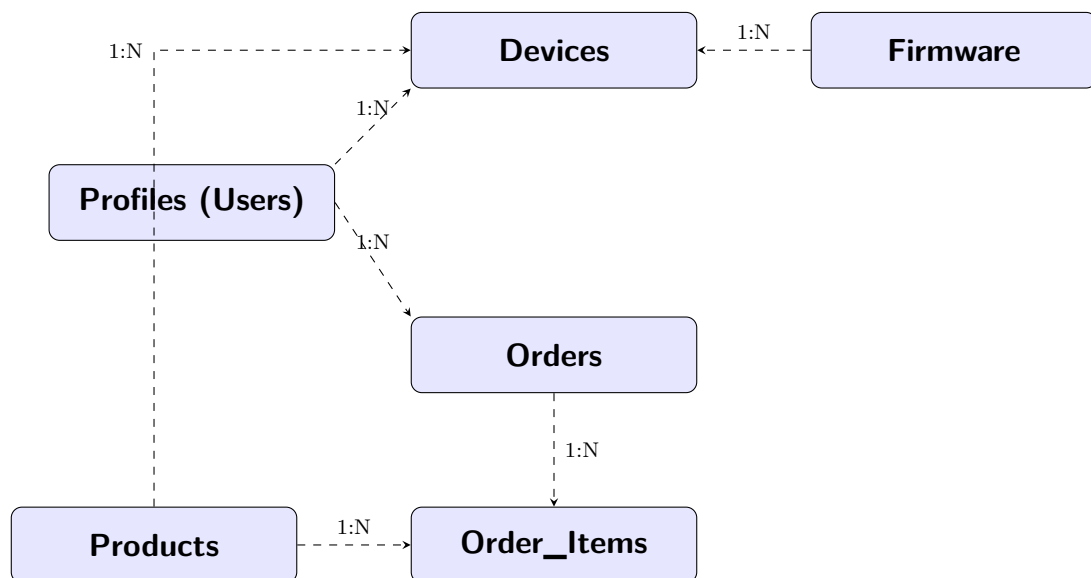


Figure 9.1: Entity-Relationship Diagram of the Supabase PostgreSQL Schema.

9.2 Table Definitions and SQL Schema

The relational schema is implemented in PostgreSQL. The ‘profiles’ table is automatically linked to Supabase’s built-in authentication system.

9.2.1 SQL Initialization Script

Listing 9.1: PostgreSQL Schema Creation Script

```

1  -- Enable UUID extension
2  CREATE EXTENSION IF NOT EXISTS "uuid-osspp";
3
4  -- Profiles (extends Supabase Auth Users)
5  CREATE TABLE profiles (
6      id UUID REFERENCES auth.users(id) PRIMARY KEY,
7      email TEXT UNIQUE NOT NULL,
8      full_name TEXT,
9      role TEXT DEFAULT 'customer' CHECK (role IN ('customer', '
      admin')),
10     created_at TIMESTAMPTZ DEFAULT NOW()
11 );
12
13 -- Products
14 CREATE TABLE products (
15     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
16     name TEXT NOT NULL,
17     description TEXT,
18     price DECIMAL(10, 2) NOT NULL,
19     image_url TEXT,
20     category TEXT,
21     stock INTEGER DEFAULT 0,
22     created_at TIMESTAMPTZ DEFAULT NOW()
23 );
24
25 -- Orders
26 CREATE TABLE orders (
27     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
28     user_id UUID REFERENCES profiles(id) NOT NULL,
29     status TEXT DEFAULT 'pending',
30     total DECIMAL(10, 2) NOT NULL,
31     shipping_address TEXT NOT NULL,
32     created_at TIMESTAMPTZ DEFAULT NOW()
33 );
34
35 -- Order Items
36 CREATE TABLE order_items (
37     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
38     order_id UUID REFERENCES orders(id) ON DELETE CASCADE,
39     product_id UUID REFERENCES products(id),

```

```

40     quantity INTEGER NOT NULL CHECK (quantity > 0),
41     price DECIMAL(10, 2) NOT NULL
42 );
43
44 -- Firmware
45 CREATE TABLE firmware (
46     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
47     version TEXT NOT NULL,
48     device_type TEXT NOT NULL,
49     download_url TEXT NOT NULL,
50     release_notes TEXT,
51     created_at TIMESTAMPTZ DEFAULT NOW()
52 );
53
54 -- Devices
55 CREATE TABLE devices (
56     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
57     serial_number TEXT UNIQUE NOT NULL,
58     user_id UUID REFERENCES profiles(id),
59     product_id UUID REFERENCES products(id),
60     firmware_version TEXT,
61     registered_at TIMESTAMPTZ DEFAULT NOW()
62 );

```

9.3 Row-Level Security (RLS) Policies

To ensure data privacy and security, Supabase's Row-Level Security (RLS) is heavily utilized. RLS ensures that users can only access or modify data that belongs to them, while administrators retain full access.

Info

Row-Level Security is a PostgreSQL feature that restricts which rows are returned by normal queries, or are permitted for data modification commands, based on the user executing the query.

9.3.1 Profiles Policy

Users can only view and update their own profiles.

```

1 ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
2 CREATE POLICY "Users can view own profile" ON profiles
3     FOR SELECT USING (auth.uid() = id);
4 CREATE POLICY "Users can update own profile" ON profiles
5     FOR UPDATE USING (auth.uid() = id);

```

9.3.2 Orders and Devices Policy

Customers can only view their own orders and registered devices.

```
1  -- Orders
2  ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
3  CREATE POLICY "Users can view own orders" ON orders
4      FOR SELECT USING (auth.uid() = user_id);
5
6  -- Devices
7  ALTER TABLE devices ENABLE ROW LEVEL SECURITY;
8  CREATE POLICY "Users can view own devices" ON devices
9      FOR SELECT USING (auth.uid() = user_id);
```

9.3.3 Public Access Policies

The `products` and `firmware` tables are publicly readable, allowing unauthenticated users to browse the catalog and devices to fetch firmware updates without complex authentication handshakes.

```
1  ALTER TABLE products ENABLE ROW LEVEL SECURITY;
2  CREATE POLICY "Products are publicly visible" ON products
3      FOR SELECT USING (true);
4
5  ALTER TABLE firmware ENABLE ROW LEVEL SECURITY;
6  CREATE POLICY "Firmware is publicly visible" ON firmware
7      FOR SELECT USING (true);
```

Chapter 10

Testing & Validation

Rigorous testing was conducted to ensure the reliability, accuracy, and robust performance of the Mountain Climber Health & GPS Tracker system under simulated field conditions. This chapter presents the methodologies and results of various hardware and software tests.

10.1 LoRa Range Testing

Given the critical nature of communication in remote areas, the SX1278 LoRa modules were tested extensively for range and signal integrity under different spreading factors (SF) and line-of-sight (LOS) conditions.

Info

Testing was conducted at 433MHz with a bandwidth of 125kHz and a coding rate of 4/5. Transmission power was set to 20 dBm.

10.1.1 Test Methodology

Two nodes were utilized: a transmitter periodically sending 50-byte packets and a receiver logging the Received Signal Strength Indicator (RSSI) and Packet Delivery Ratio (PDR). Tests were performed in an open field (LOS) and a moderately wooded suburban area (Non-LOS).

10.1.2 Results

The results in Table 10.1 indicate that while SF7 provides adequate LOS range, utilizing higher spreading factors like SF10 or SF12 is crucial for maintaining low packet loss in non-LOS mountainous environments, albeit at the cost of lower data rates.

10.2 Sensor Accuracy Validation



Figure 10.1: LoRa Range Testing Setup in Field Conditions

Table 10.1: LoRa Range Testing Results (Transmitter Power: 20 dBm, BW: 125kHz)

Environment	Spreading Factor	Distance (km)	Avg RSSI (dBm)	Packet Loss (%)
Line-of-Sight (LOS)	SF7	1.5	-95	0.0
Line-of-Sight (LOS)	SF7	3.0	-108	2.1
Line-of-Sight (LOS)	SF10	5.0	-115	1.5
Non-LOS (Wooded)	SF7	0.5	-102	4.2
Non-LOS (Wooded)	SF7	1.2	-118	22.5
Non-LOS (Wooded)	SF10	1.5	-112	3.0
Non-LOS (Wooded)	SF12	2.5	-120	8.5

10.2.1 MAX30102 Health Sensor

The MAX30102 sensor's heart rate (BPM) and blood oxygen saturation (SpO2) readings were validated against a medical-grade finger pulse oximeter (ChoiceMMed MD300C2).

Table 10.2: MAX30102 Accuracy Comparison (Resting State, N=50 samples)

Metric	Reference Device (Avg)	MAX30102 (Avg)	Mean Absolute Error
Heart Rate (BPM)	72.4	73.1	1.8 BPM
SpO2 (%)	98.2	97.5	0.9 %

Warning

While resting accuracy is within acceptable margins, vigorous wrist movement introduces motion artifacts, occasionally resulting in SpO2 reading dropouts.

10.2.2 NEO-6M GPS Module

GPS accuracy was tested by placing the module at known surveyed coordinates. The module achieved a typical horizontal position error of 2.5 to 4 meters under clear skies (Time-To-First-Fix: ~35 seconds from cold start).

10.3 BLE Connection Reliability

The BLE link between the Wristband/Climber Main device and the Flutter mobile application was tested for connection stability and automatic reconnection.

- **Connection Time:** Average 1.2 seconds to establish connection and discover GATT services.
- **Range:** Reliable connection maintained up to 8 meters indoors and 15 meters outdoors.
- **Reconnection Behavior:** When the connection drops due to distance, the Flutter app successfully reconnects automatically within 3 seconds of re-entering the range.

10.4 System Integration Testing

An end-to-end integration test was conducted to verify the complete data pipeline from the climber to the base camp dashboard.

Test Scenario: SOS Emergency Trigger

1. **Trigger:** Climber presses the physical SOS button on the Main Device.
2. **Local Action:** Device buzzer activates, RGB LED turns red, and the Flutter app immediately displays a local emergency warning via BLE.
3. **LoRa Transmission:** An SOS packet (Type 0x01) containing current GPS coordinates is broadcast via LoRa.
4. **Relay:** The intermediate Repeater node successfully receives, validates the CRC, and re-broadcasts the packet.
5. **Reception:** Base Camp unit receives the packet, outputs JSON over serial to the connected laptop.
6. **Dashboard Alert:** The Flask dashboard parses the JSON, plays a loud audio alarm, flashes the screen red, and updates the climber's marker on the real-time map to indicate an emergency state.

Result: The entire pipeline, from button press to dashboard alarm, consistently executed in under 1.5 seconds.

Chapter 11

Budget & Bill of Materials

This chapter details the financial planning for the Mountain Climber Health & GPS Tracker. The budget is divided into hardware component costs (Bill of Materials) and general development and deployment costs. All prices are estimated in Sri Lankan Rupees (LKR) based on local electronics suppliers at the time of development.

11.1 Device Bill of Materials (BOM)

The system consists of three distinct hardware units. Table 11.1 outlines the components required for a single Climber Main Device, which is the most complex unit in the system.

Table 11.1: Bill of Materials for a Single Climber Main Device

Component	Specification	Qty	Unit Cost (LKR)	Total (LKR)
Microcontroller	ESP32 DevKit V1	1	2,500	2,500
Health Sensor	MAX30102 Pulse Oximeter	1	800	800
GPS Module	Ublox NEO-6M	1	1,500	1,500
LoRa Transceiver	SX1278 (433MHz)	1	1,200	1,200
Power Supply	18650 Li-ion Battery (3.7V)	1	600	600
Charging Module	TP4056 Battery Charger	1	250	250
Enclosure	Custom 3D Printed / ABS	1	500	500
Miscellaneous	PCBs, Wires, Connectors, LEDs	1	800	800
Total Cost per Climber Device				8,150

Info

The Wristband, Repeater, and Base Camp units utilize subsets of these components. For example, a Repeater only requires an ESP32, SX1278, battery, and enclosure, bringing its unit cost down to approximately LKR 5,650.

11.2 Total Hardware System Cost

For a standard expedition setup, the system is designed to support multiple climbers and require relay nodes depending on the terrain. Table 11.2 calculates the cost for a typical base configuration consisting of two climbers, one repeater, and one base camp unit.

Table 11.2: Hardware Cost for a Standard Expedition Setup

Unit Type	Estimated Unit Cost (LKR)	Quantity	Total (LKR)
Climber Main Device	8,150	2	16,300
Wristband Device (BLE)	4,500	2	9,000
Repeater Node	5,650	1	5,650
Base Camp Unit	5,200	1	5,200
Total Hardware System Cost			36,150

11.3 Software and Development Costs

While the system heavily utilizes open-source software (Flutter, Python, PostgreSQL), there are infrastructural costs associated with hosting the web portal, database, and domain names.

Table 11.3: Development and Infrastructure Costs

Item	Description	Cost (LKR)
Domain Name	summitgear.com (Annual)	4,500
Web Hosting (Vercel)	Next.js frontend hosting	Free Tier
Database Hosting (Supabase)	PostgreSQL, Auth, Storage	Free Tier
Development Tools	Soldering supplies, multimeters	3,500
Prototyping Waste	Damaged/burnt components during R&D	2,500
Total Development Costs		10,500

11.4 Budget Summary

The grand total for deploying the prototype system, including a standard hardware configuration and initial infrastructure setup, is calculated as follows:

Table 11.4: Grand Total Budget Summary

Category	Amount (LKR)
Hardware System Cost	36,150
Development & Infrastructure Costs	10,500
Contingency Fund (10%)	4,665
Grand Total Estimate	51,315

Chapter 12

Limitations & Future Work

While the current prototype of the Mountain Climber Health & GPS Tracker successfully demonstrates the core functionalities of off-grid safety monitoring, several limitations were identified during development and testing. This chapter outlines these constraints and proposes future enhancements for a production-ready system.

12.1 Known Limitations

- **GPS Accuracy in Dense Canopy:** The Ublox NEO-6M module struggles to acquire a fast and accurate satellite fix under dense forest canopies or deep ravines, potentially delaying location updates during an emergency.
- **LoRa Line-of-Sight Dependency:** While 433MHz penetrates vegetation better than 2.4GHz, extreme geological features (steep valleys, massive rock faces) severely attenuate the signal, necessitating careful, manual placement of Repeater nodes.
- **Motion Artifacts on MAX30102:** The optical pulse oximetry sensor is highly sensitive to ambient light and movement. Vigorous climbing motions can introduce noise, leading to temporarily inaccurate heart rate or SpO2 readings.
- **Single-Channel LoRa Limitations:** The system currently uses a single LoRa channel and spreading factor. It lacks frequency hopping or adaptive data rates, meaning network collisions can occur if many devices transmit simultaneously in a dense deployment.
- **Battery Constraints:** Constantly polling GPS and listening on the LoRa radio draws significant current. Without power optimization, the 18650 battery life is limited to approximately 18-24 hours of continuous operation.

12.2 Future Improvements

To address these limitations and commercialize the system, the following future works are proposed:

12.2.1 Hardware Enhancements

- **Multi-channel LoRa Gateway:** Upgrading the Base Camp unit to a multi-channel LoRaWAN gateway (e.g., using SX1302) to handle hundreds of concurrent nodes and implement Adaptive Data Rate (ADR) protocols.
- **Solar Energy Harvesting:** Integrating small, flexible solar panels into the device enclosures or backpacks to trickle-charge the lithium batteries, significantly extending deployment life.
- **Satellite Communication Fallback:** Integrating an Iridium or Swarm satellite modem for extreme emergencies when the climber falls completely outside the LoRa mesh network range.
- **IP67 Waterproofing:** Designing injection-molded enclosures with silicone gaskets to achieve true IP67 weather resistance for harsh alpine environments.

12.2.2 Software & System Enhancements

- **Mesh Routing Protocol:** Implementing an intelligent, self-healing mesh routing protocol (like Reticulum or Meshtastic's underlying logic) to replace the current static repeater logic.
- **Machine Learning Health Predictions:** Developing ML models that run on the dashboard to analyze vitals trends over time, predicting altitude sickness or exhaustion before acute symptoms appear.
- **Cloud Dashboard Sync:** Developing a mechanism for the local Base Camp dashboard to opportunistically sync its data to the Next.js cloud portal whenever an intermittent internet connection (e.g., via Starlink or cellular) is available.

Appendix A

Pin Configuration Tables

This appendix provides the complete General Purpose Input/Output (GPIO) mapping for all microcontrollers utilized in the Mountain Climber Health & GPS Tracker system.

A.1 Climber Main Device (ESP32)

GPIO Number	Connected To	Function	Protocol
GPIO 5	SX1278 NSS	LoRa Chip Select	SPI
GPIO 18	SX1278 SCK	SPI Clock	SPI
GPIO 19	SX1278 MISO	SPI Master In Slave Out	SPI
GPIO 23	SX1278 MOSI	SPI Master Out Slave In	SPI
GPIO 14	SX1278 RST	LoRa Reset	Digital Out
GPIO 26	SX1278 DIO0	LoRa Interrupt	Digital In
GPIO 21	MAX30102 SDA	I2C Data	I2C
GPIO 22	MAX30102 SCL	I2C Clock	I2C
GPIO 16	NEO-6M TX	GPS Data Transmit	UART2 RX
GPIO 17	NEO-6M RX	GPS Data Receive	UART2 TX
GPIO 4	Active Buzzer	Audio Alarm	Digital Out/PWM
GPIO 25	RGB LED Red	Visual Status Indicator	Digital Out
GPIO 33	RGB LED Green	Visual Status Indicator	Digital Out
GPIO 32	RGB LED Blue	Visual Status Indicator	Digital Out
GPIO 13	Tactile Push Button	SOS Trigger	Digital In (Pull-up)

A.2 Wristband Device (ESP32-C3 / PICO)

A.3 Repeater Node (ESP32)

A.4 Base Camp Unit (ESP32)

GPIO Number	Connected To	Function	Protocol
GPIO 21	MAX30102 SDA	I2C Data	I2C
GPIO 22	MAX30102 SCL	I2C Clock	I2C
GPIO 9	Battery Voltage ADC	Power Monitoring	Analog In
GPIO 4	Haptic Motor	Vibration Alert	Digital Out/PWM
GPIO 5	System LED	Status Indicator	Digital Out

GPIO Number	Connected To	Function	Protocol
GPIO 5	SX1278 NSS	LoRa Chip Select	SPI
GPIO 18	SX1278 SCK	SPI Clock	SPI
GPIO 19	SX1278 MISO	SPI Master In Slave Out	SPI
GPIO 23	SX1278 MOSI	SPI Master Out Slave In	SPI
GPIO 14	SX1278 RST	LoRa Reset	Digital Out
GPIO 26	SX1278 DIO0	LoRa Interrupt	Digital In
GPIO 2	Status LED	Node Active Indicator	Digital Out

GPIO Number	Connected To	Function	Protocol
GPIO 5	SX1278 NSS	LoRa Chip Select	SPI
GPIO 18	SX1278 SCK	SPI Clock	SPI
GPIO 19	SX1278 MISO	SPI Master In Slave Out	SPI
GPIO 23	SX1278 MOSI	SPI Master Out Slave In	SPI
GPIO 14	SX1278 RST	LoRa Reset	Digital Out
GPIO 26	SX1278 DIO0	LoRa Interrupt	Digital In
TX0 (GPIO 1)	PC USB Serial	JSON Data to Dashboard	UART0
RX0 (GPIO 3)	PC USB Serial	Commands from Dashboard	UART0

Appendix B

LoRa Packet Format Specification

To optimize airtime and maximize the range of the LoRa transmissions, a custom, lightweight binary packet protocol is utilized instead of transmitting verbose string formats. This appendix details the byte-level structure.

B.1 General Packet Structure

Every transmission adheres to a standard header format followed by a variable-length payload.

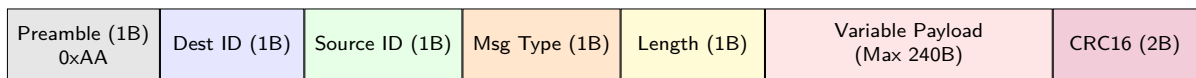


Figure B.1: Binary LoRa Packet Structure

B.2 Message Types and Payload Definitions

The `Msg Type` byte dictates how the receiver should parse the payload.

B.2.1 0x01: SOS Alert

Transmitted when a climber triggers an emergency.

- **Payload Length:** 8 bytes
- **Structure:** [Latitude (4B Float)] [Longitude (4B Float)]

B.2.2 0x02: Routine GPS Update

Periodic location update.

- **Payload Length:** 8 bytes
- **Structure:** [Latitude (4B Float)] [Longitude (4B Float)]

B.2.3 0x03: Health Vitals

Periodic or on-demand health data.

- **Payload Length:** 3 bytes
- **Structure:** [Heart Rate BPM (1B Uint)] [SpO2 % (1B Uint)] [Temperature C (1B Int)]

B.2.4 0x04: Text Message

Two-way chat messages.

- **Payload Length:** 1 to 240 bytes
- **Structure:** [ASCII Text String]

B.3 Example Hex Dumps

Example 1: Health Vitals Update Climber 0x05 sends health data to Base Camp 0x00. HR=85, SpO2=98, Temp=24.

AA 00 05 03 03 55 62 18 [CRC_H] [CRC_L]

Breakdown: AA (Preamble), 00 (Dest Base), 05 (Source Climber), 03 (MsgType Health), 03 (Len 3), 55 (85 BPM), 62 (98%), 18 (24C).

Example 2: SOS Alert Climber 0x02 triggers SOS with coordinates 7.2906, 80.6337.

AA 00 02 01 08 40 E9 4B 38 42 A1 44 80 [CRC_H] [CRC_L]

Breakdown: The 8-byte payload represents the two 32-bit IEEE 754 floats for Lat and Lng.

Appendix C

API Reference

This appendix documents the key Application Programming Interfaces (APIs) used for communication between the Flask Base Camp Dashboard and the Next.js Supabase web portal.

C.1 Base Camp Dashboard (Flask REST API)

The Python Flask dashboard exposes a local API that the desktop UI interacts with to retrieve serial data and send commands.

C.1.1 GET /

Renders the main dashboard HTML interface.

- **Authentication:** None (Localhost only)
- **Response:** HTML page.

C.1.2 GET /get_data

Polls the latest telemetry data received from the serial port.

- **Authentication:** None
- **Response:** JSON Array of climber objects.

```
1 {  
2   "climbers": [  
3     {  
4       "id": 5,  
5       "lat": 7.2906,  
6       "lng": 80.6337,  
7       "hr": 85,  
8       "spo2": 98,  
9       "status": "normal",  
10      "last_seen": "2024-05-20T10:30:00Z"
```

```

11     }
12   ]
13 }

```

C.1.3 POST /send_message

Queues a text message to be transmitted via LoRa to a specific climber.

- **Request Body (JSON):**

```

1   {
2     "dest_id": 5,
3     "message": "Weather deteriorating. Return to base."
4   }

```

- **Response:** HTTP 200 OK

C.1.4 GET /export_csv

Triggers a download of all logged telemetry data for post-expedition analysis.

- **Response:** text/csv file download.

C.2 Summit Gear Web Portal (Next.js Server Actions)

The Next.js portal utilizes Server Actions for interacting with the Supabase database. These act as secure, server-side API endpoints.

C.2.1 registerDevice(serialNum, userId)

Registers a new physical device to a user's account.

- **Auth Required:** Yes (Valid session JWT)
- **Response:** Success boolean or error message.

C.2.2 createOrder(cart: CartItem[], userId: string)

Processes an e-commerce checkout.

- **Auth Required:** Yes
- **Response:** Order UUID string.

C.2.3 `getProducts(category?: string)`

Fetches the catalog of available gear and devices.

- **Auth Required:** No
- **Response:** Array of Product objects.

C.2.4 `uploadFirmware(file: FormData, version: string)`

Admin-only endpoint to upload a new OTA firmware binary.

- **Auth Required:** Yes (Role must be 'admin')
- **Response:** Download URL of the uploaded asset in Supabase Storage.